

## ✓ Sales Performance Analysis

### Data Analytics Internship Project

---

 Submitted By

**Rudrashetty Nandini**

 Internship Domain

**Data Analytics**

 Project Objective

This project focuses on analyzing sales data to identify business trends, regional performance, profitable product categories, and seasonal sales patterns. The goal is to generate actionable business insights using data analytics and visualization techniques.

---

 Tools & Technologies Used

- Python
  - Pandas
  - NumPy
  - Matplotlib
  - Seaborn
  - Google Colab
- 

 Dataset

**Superstore Sales Dataset from Kaggle**

---

 Key Analysis Performed

- Data Cleaning & Preprocessing
  - KPI Analysis
  - Monthly Sales Trend Analysis
  - Regional Sales Analysis
  - Category-wise Performance
  - Top & Worst Performing Products
  - Profitability Analysis
  - Seasonal Sales Analysis
  - Business Recommendations
- 

 Expected Outcome

The analysis helps understand customer purchasing behavior, improve business decision-making, optimize inventory planning, and increase profitability through data-driven insights.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_style("whitegrid")
```

```
df = pd.read_csv('/content/superstore_final_dataset (1).csv', encoding='latin1')
```

```
df.head()
```

|   | Row_ID | Order_ID       | Order_Date | Ship_Date  | Ship_Mode      | Customer_ID | Customer_Name   | Segment   | Country       | City            | State      | Postal_Code |
|---|--------|----------------|------------|------------|----------------|-------------|-----------------|-----------|---------------|-----------------|------------|-------------|
| 0 | 1      | CA-2017-152156 | 8/11/2017  | 11/11/2017 | Second Class   | CG-12520    | Claire Gute     | Consumer  | United States | Henderson       | Kentucky   | 40502       |
| 1 | 2      | CA-2017-152156 | 8/11/2017  | 11/11/2017 | Second Class   | CG-12520    | Claire Gute     | Consumer  | United States | Henderson       | Kentucky   | 40502       |
| 2 | 3      | CA-2017-138688 | 12/6/2017  | 16/06/2017 | Second Class   | DV-13045    | Darrin Van Huff | Corporate | United States | Los Angeles     | California | 90008       |
| 3 | 4      | US-2016-108966 | 11/10/2016 | 18/10/2016 | Standard Class | SO-20335    | Sean O Donnel   | Consumer  | United States | Fort Lauderdale | Florida    | 33304       |
| 4 | 5      | US-2016-108966 | 11/10/2016 | 18/10/2016 | Standard Class | SO-20335    | Sean O Donnel   | Consumer  | United States | Fort Lauderdale | Florida    | 33304       |

```
df.shape
```

```
(9800, 18)
```

```
df.columns
```

```
Index(['Row_ID', 'Order_ID', 'Order_Date', 'Ship_Date', 'Ship_Mode',
       'Customer_ID', 'Customer_Name', 'Segment', 'Country', 'City', 'State',
       'Postal_Code', 'Region', 'Product_ID', 'Category', 'Sub_Category',
       'Product_Name', 'Sales'],
      dtype='object')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9800 entries, 0 to 9799
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   Row_ID                9800 non-null  int64
1   Order_ID              9800 non-null  object
2   Order_Date            9800 non-null  object
3   Ship_Date             9800 non-null  object
4   Ship_Mode             9800 non-null  object
5   Customer_ID           9800 non-null  object
6   Customer_Name         9800 non-null  object
7   Segment               9800 non-null  object
8   Country               9800 non-null  object
9   City                  9800 non-null  object
10  State                 9800 non-null  object
11  Postal_Code           9789 non-null  float64
12  Region                9800 non-null  object
13  Product_ID            9800 non-null  object
14  Category              9800 non-null  object
15  Sub_Category          9800 non-null  object
16  Product_Name          9800 non-null  object
17  Sales                 9800 non-null  float64
dtypes: float64(2), int64(1), object(15)
memory usage: 1.3+ MB
```

```
df.describe()
```

|       | Row_ID      | Postal_Code  | Sales        |
|-------|-------------|--------------|--------------|
| count | 9800.000000 | 9789.000000  | 9800.000000  |
| mean  | 4900.500000 | 55273.322403 | 230.769059   |
| std   | 2829.160653 | 32041.223413 | 626.651875   |
| min   | 1.000000    | 1040.000000  | 0.444000     |
| 25%   | 2450.750000 | 23223.000000 | 17.248000    |
| 50%   | 4900.500000 | 58103.000000 | 54.490000    |
| 75%   | 7350.250000 | 90008.000000 | 210.605000   |
| max   | 9800.000000 | 99301.000000 | 22638.480000 |

```
df.isnull().sum()
```

|               | 0  |
|---------------|----|
| Row_ID        | 0  |
| Order_ID      | 0  |
| Order_Date    | 0  |
| Ship_Date     | 0  |
| Ship_Mode     | 0  |
| Customer_ID   | 0  |
| Customer_Name | 0  |
| Segment       | 0  |
| Country       | 0  |
| City          | 0  |
| State         | 0  |
| Postal_Code   | 11 |
| Region        | 0  |
| Product_ID    | 0  |
| Category      | 0  |
| Sub_Category  | 0  |
| Product_Name  | 0  |
| Sales         | 0  |

dtype: int64

```
df.duplicated().sum()
```

```
np.int64(0)
```

## ✓ Sales Performance Analysis

### Objective

Analyze sales performance across regions, categories and time.

### Data Cleaning

### KPI Analysis

### Visualizations

### Insights

### Recommendations

```
df['Order_Date'] = pd.to_datetime(df['Order_Date'], dayfirst=True)
```

```
df['Year'] = df['Order_Date'].dt.year
df['Month'] = df['Order_Date'].dt.month
df['Month_Name'] = df['Order_Date'].dt.month_name()
```

## ▼ KPI Analysis

### ▼ KPI 1 — Total Sales

```
total_sales = df['Sales'].sum()

print("Total Sales:", total_sales)

Total Sales: 2261536.7827000003
```

### ▼ KPI 2 — Total Orders

```
total_orders = df['Order_ID'].nunique()

print("Total Orders:", total_orders)

Total Orders: 4922
```

### ▼ KPI 3 — Total Customers

```
total_customers = df['Customer_ID'].nunique()

print("Total Customers:", total_customers)

Total Customers: 793
```

### KPI 4 — Average Order Value

Formula:

$$\text{Average Order Value} = \frac{\text{Total Sales}}{\text{Number of Orders}}$$

## ▼ Insight

The KPI analysis provides a quick overview of the company's financial performance. Total sales indicate overall revenue generation, while profit margin helps measure operational efficiency and profitability.

```
aov = df['Sales'].sum() / df['Order_ID'].nunique()

print("Average Order Value:", round(aov,2))

Average Order Value: 459.48
```

Note: The dataset did not include profit-related information. Therefore, the analysis primarily focused on sales trends, order patterns, customer distribution, and regional performance.

## ▼ Sales Trend Analysis

```
monthly_sales = df.groupby('Month_Name')['Sales'].sum()

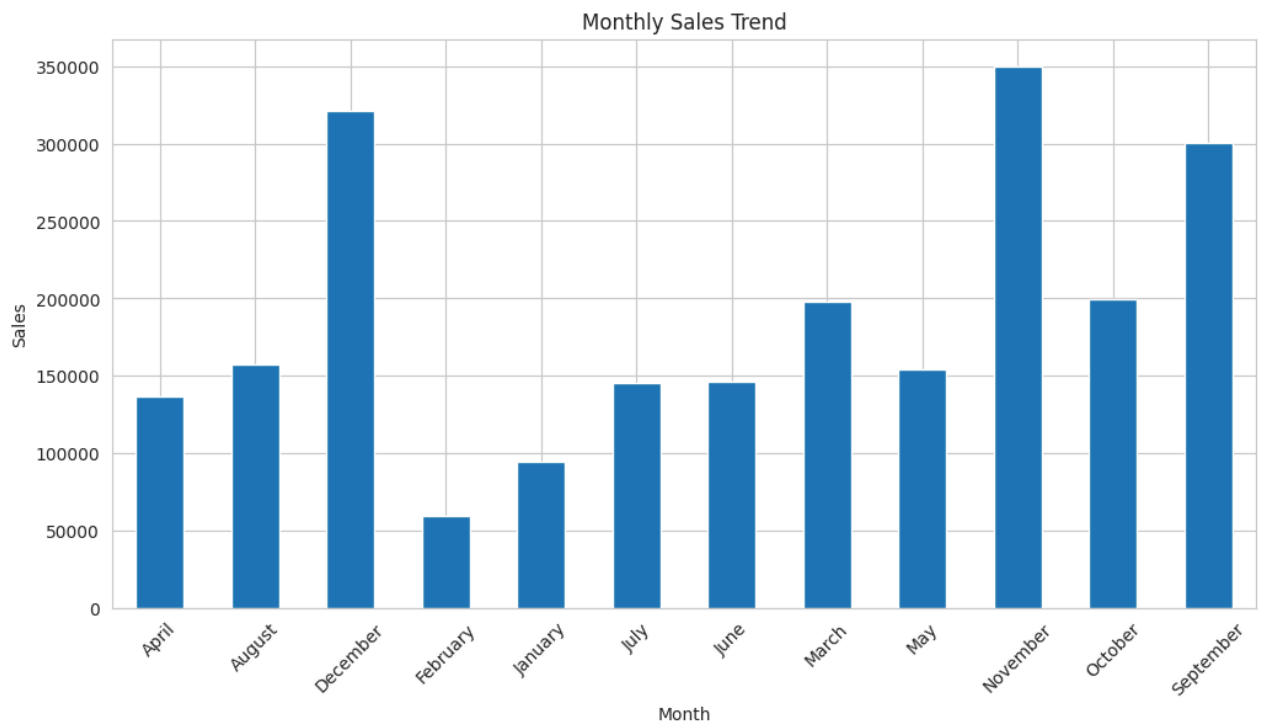
plt.figure(figsize=(12,6))

monthly_sales.plot(kind='bar')
```

```
plt.title('Monthly Sales Trend')
plt.xlabel('Month')
plt.ylabel('Sales')

plt.xticks(rotation=45)

plt.show()
```



## Insight

The monthly sales trend shows fluctuations in revenue across different months. Certain months generated significantly higher sales, indicating possible seasonal demand and peak purchasing periods.

## Region-wise Sales Analysis

```
region_sales = df.groupby('Region')['Sales'].sum()

plt.figure(figsize=(8,8))

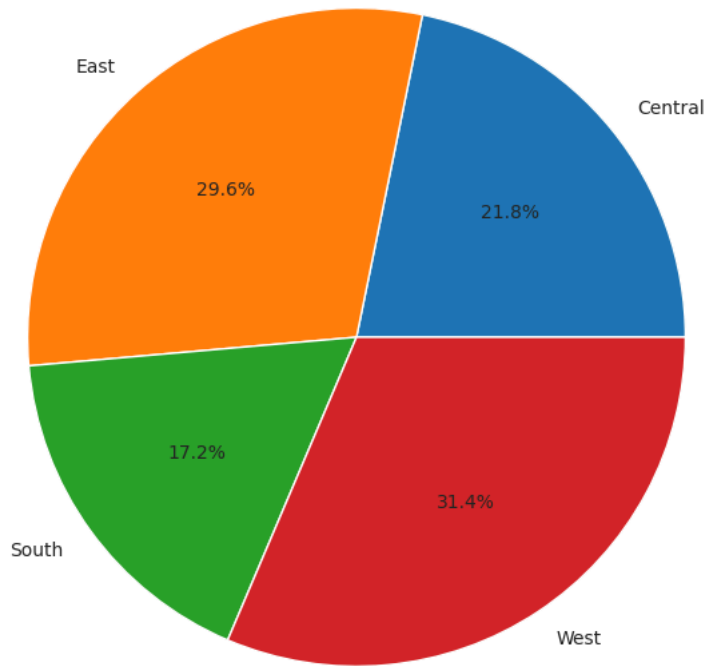
region_sales.plot(kind='pie', autopct='%1.1f%%')

plt.title('Sales by Region')

plt.ylabel('')

plt.show()
```

Sales by Region

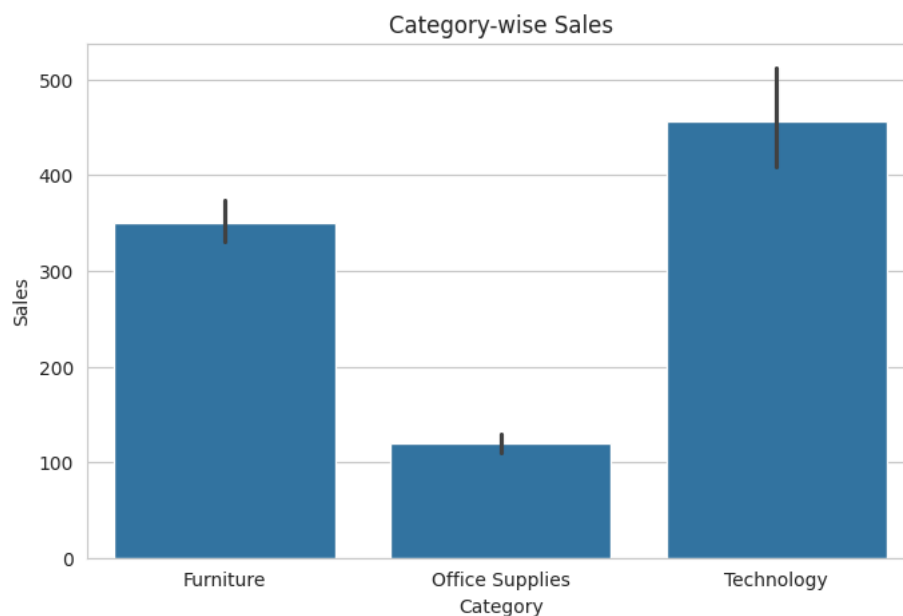


### Insight

Regional analysis reveals that some regions contribute more revenue than others. High-performing regions may have stronger customer demand, while low-performing regions may require targeted marketing and promotional strategies.

### Category-wise Sales

```
plt.figure(figsize=(8,5))  
  
sns.barplot(x='Category', y='Sales', data=df)  
  
plt.title('Category-wise Sales')  
  
plt.show()
```



## Insight

The category analysis shows which product categories dominate overall sales. Categories with higher contribution indicate strong customer preference and market demand.

### Top 10 Selling Products

```
top_products = df.groupby('Sub_Category')['Sales'].sum().sort_values(ascending=False).head(10)

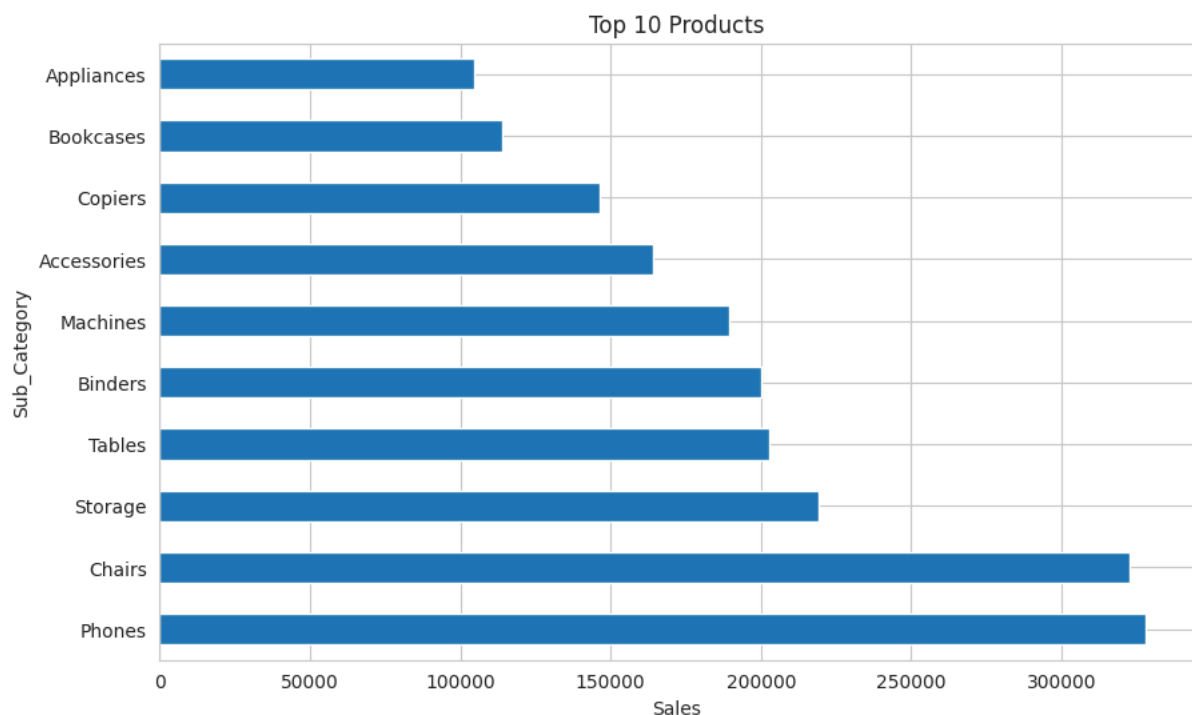
plt.figure(figsize=(10,6))

top_products.plot(kind='barh')

plt.title('Top 10 Products')

plt.xlabel('Sales')

plt.show()
```



## Business Insight

The analysis identifies the best-selling sub-categories driving revenue growth. Maintaining product availability and customer engagement for these products can improve long-term sales performance.

### Heatmap Analysis

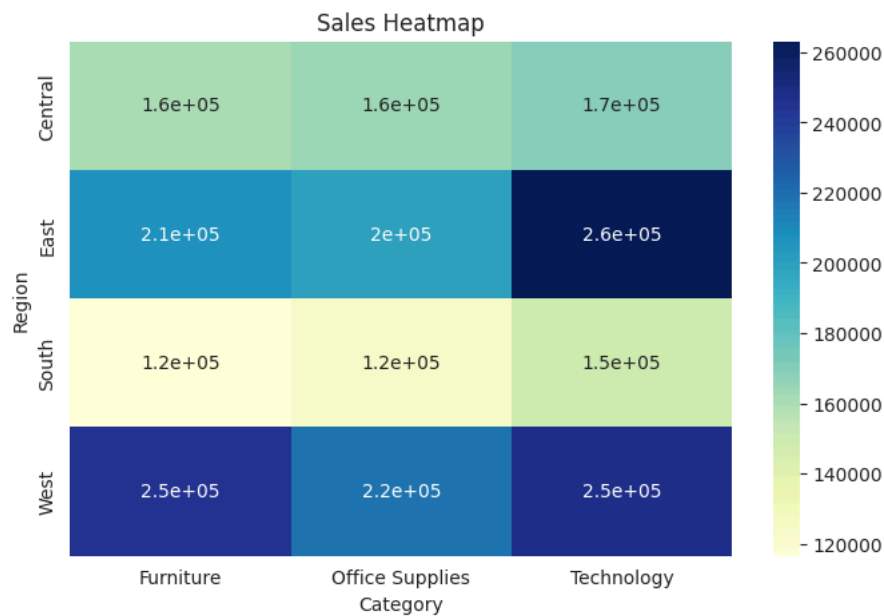
```
pivot = df.pivot_table(
    values='Sales',
    index='Region',
    columns='Category',
    aggfunc='sum'
)

plt.figure(figsize=(8,5))

sns.heatmap(pivot, annot=True, cmap='YlGnBu')

plt.title('Sales Heatmap')

plt.show()
```



### Insight

The correlation analysis helps identify relationships between numerical variables such as sales, profit, quantity, and discounts. Strong correlations can support better business decision-making and forecasting.

### ▼ Seasonality Analysis

```
seasonality = df.groupby('Month_Name')['Sales'].sum()

plt.figure(figsize=(12,5))

seasonality.plot(marker='o')

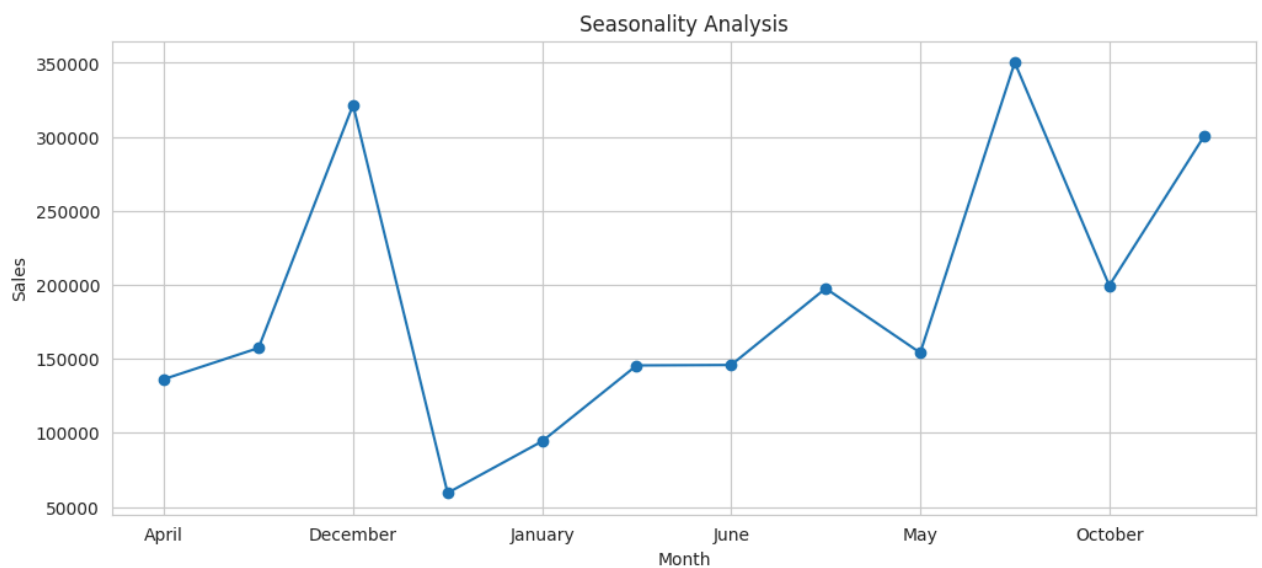
plt.title('Seasonality Analysis')

plt.xlabel('Month')

plt.ylabel('Sales')

plt.grid(True)

plt.show()
```



### Business Insight



Seasonal analysis indicates recurring sales patterns throughout the year. Understanding these trends helps businesses optimize inventory planning, staffing, and marketing strategies.

## Customer Segment Analysis

```
segment_sales = df.groupby('Segment')['Sales'].sum()

plt.figure(figsize=(8,5))

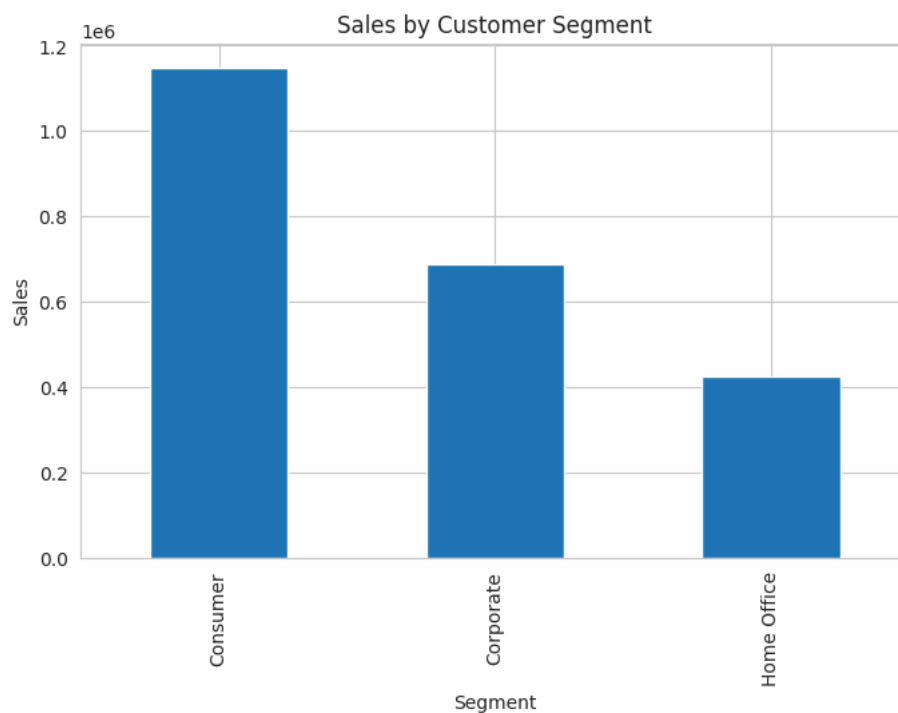
segment_sales.plot(kind='bar')

plt.title('Sales by Customer Segment')

plt.xlabel('Segment')

plt.ylabel('Sales')

plt.show()
```



## State-wise Sales Analysis

```
top_states = df.groupby('State')['Sales'].sum().sort_values(ascending=False).head(10)

plt.figure(figsize=(10,6))

top_states.plot(kind='bar')

plt.title('Top 10 States by Sales')

plt.xlabel('State')

plt.ylabel('Sales')

plt.xticks(rotation=45)

plt.show()
```

